Mock HCM Server - Specification

1. Purpose

The Mock HCM Server simulates Human Capital Management system APIs that ReadyOn's Time-Off Microservice integrates with. It provides configurable behaviors to test various scenarios:

- Normal operations
- Network latency and timeouts
- Partial failures
- Balance changes (work anniversaries, corrections)
- Invalid dimensions
- Insufficient balance scenarios
- Circuit breaker activation

---

2. Architecture

```
Mock HCM Server (Express/NestJS on port 3001)
├── Controllers
│   ├── BalanceController
│   ├── RequestController
│   └── SyncController
├── Services
│   ├── BalanceService
│   ├── RequestService
│   └── BehaviorEngineService
├── Behaviors
│   ├── NormalBehavior
│   ├── TimeoutBehavior
│   ├── PartialFailureBehavior
│   ├── DriftBehavior
│   ├── CircuitBreakerBehavior
│   └── InvalidDimensionBehavior
└── Data Store (in-memory JSON)
```

---

3. Endpoints

3.1 Get Balance (Real-Time)

Endpoint: `GET /hcm/balances/{employeeId}/{locationId}`

Query Parameters:

- `leaveType` (required): VACATION, SICK, PTO, etc.

Response (200 OK):

```
{
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "availableBalance": 10.0,
  "usedBalance": 5.0,
  "carryoverBalance": 2.0,
  "balanceYear": 2026,
  "lastUpdatedAt": "2026-04-24T10:00:00Z"
}
```

Error Responses:

- `400 Bad Request` : Missing or invalid parameters
- `404 Not Found` : Employee or location not found in HCM
- `500 Internal Server Error` : Simulated HCM error (when behavior activated)
- `503 Service Unavailable` : HCM down (circuit breaker behavior)
- `504 Gateway Timeout` : Response delayed > 5 seconds (timeout behavior)

3.2 Submit Time-Off Request

Endpoint: `POST /hcm/time-off-requests`

Request Body:

```
{
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "startDate": "2026-05-01",
  "endDate": "2026-05-03",
  "days": 3.0,
  "requestId": "REQ-001"
}
```

Response (200 OK):

```
{
  "hcmRequestId": "HCM-98765",
  "status": "APPROVED",
  "approvedAt": "2026-04-24T10:00:00Z"
}
```

Error Responses:

- `400 Bad Request`: Invalid request format
- `403 Forbidden`: Insufficient balance
- `404 Not Found`: Invalid employee-location-leaveType combination
- `409 Conflict`: Duplicate submission (same request ID)
- `500 Internal Server Error`: Simulated error

3.3 Batch Sync Balances

Endpoint: `POST /hcm/sync/balances`

Request Body:

```
{
  "employeeIds": ["EMP-001", "EMP-002", "EMP-003"]
}
```

Response (200 OK):

```
{
  "balances": [
    {
      "employeeId": "EMP-001",
      "balances": [
        {
          "locationId": "LOC-NYC",
          "leaveType": "VACATION",
          "availableBalance": 10.0,
          "usedBalance": 5.0,
          "carryoverBalance": 2.0,
          "balanceYear": 2026,
          "lastUpdatedAt": "2026-04-24T10:00:00Z"
        }
      ]
    }
  ],
  "failedEmployees": [
    {
      "employeeId": "EMP-999",
      "reason": "Employee not found in HCM"
    }
  ]
}
```

3.4 Admin: Reset to Normal Behavior

Endpoint: `POST /hcm/admin/reset-behavior`

Request Body:

```
{}
```

Response (200 OK):

```
{
  "message": "Behavior reset to NORMAL"
}
```

3.5 Admin: Activate Behavior

Endpoint: `POST /hcm/admin/set-behavior`

Request Body:

```
{
  "behavior": "TIMEOUT",
  "config": {
    "delayMs": 6000
  }
}
```

Supported Behaviors:

- `NORMAL` : All requests succeed normally
- `TIMEOUT` : Respond after 6+ seconds (simulates timeout)
- `PARTIAL_FAILURE` : First request fails, subsequent succeed
- `DRIFT` : Change balance on subsequent calls (work anniversary)
- `CIRCUIT_BREAKER` : Fail first N calls, then succeed
- `INVALID_DIMENSION` : Return 404 for specific employee
- `INSUFFICIENT_BALANCE` : Return 403 for specific request

3.6 Admin: Update Balance

Endpoint: `POST /hcm/admin/update-balance`

Request Body:

```
{
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "availableBalance": 15.0,
  "usedBalance": 5.0,
  "carryoverBalance": 2.0
}
```

Response (200 OK):

```
{
  "message": "Balance updated",
  "updatedBalance": {
    "employeeId": "EMP-001",
    "locationId": "LOC-NYC",
    "leaveType": "VACATION",
    "availableBalance": 15.0,
    "lastUpdatedAt": "2026-04-24T11:00:00Z"
  }
}
```

3.7 Admin: Get Current State

Endpoint: `GET /hcm/admin/state`

Response (200 OK):

```
{
  "currentBehavior": "NORMAL",
  "behaviorConfig": {},
  "dataStore": {
    "employees": [
      {
        "employeeId": "EMP-001",
        "balances": [...]
      }
    ]
  }
}
```

---

4. Behaviors Implementation

4.1 NORMAL Behavior

Description: All requests succeed, balances returned as configured

Implementation:

```
async getBalance(employeeId, locationId, leaveType) {
  const balance = this.dataStore.getBalance(employeeId, locationId, leaveType);
  if (!balance) throw new NotFoundError();
  return {
    ...balance,
    lastUpdatedAt: new Date().toISOString()
  };
}
```

Test Use: Happy path testing

4.2 TIMEOUT Behavior

Description: Respond after 6+ seconds (triggers ReadyOn timeout at 5s)

Configuration:

```
{
  "behavior": "TIMEOUT",
  "config": {
    "delayMs": 6000
  }
}
```

Implementation:

```
async getBalance(employeeId, locationId, leaveType) {
  await delay(this.config.delayMs);
  const balance = this.dataStore.getBalance(employeeId, locationId, leaveType);
  if (!balance) throw new NotFoundError();
  return balance;
}
```

Test Use:

- Retry mechanism testing
- Timeout error handling
- Fallback to cached data

4.3 PARTIAL_FAILURE Behavior

Description: First N calls fail, subsequent succeed

Configuration:

```
{
  "behavior": "PARTIAL_FAILURE",
  "config": {
    "failCount": 1,
    "errorCode": 500
  }
}
```

Implementation:

```
async getBalance(employeeId, locationId, leaveType) {
  if (this.callCount < this.config.failCount) {
    this.callCount++;
    throw new InternalServerError();
  }
  const balance = this.dataStore.getBalance(employeeId, locationId, leaveType);
  if (!balance) throw new NotFoundError();
  return balance;
}
```

Test Use:

- Retry logic validation
- Eventual success after transient failures

4.4 DRIFT Behavior

Description: Balance changes between calls (simulates work anniversary bonus)

Configuration:

```
{
  "behavior": "DRIFT",
  "config": {
    "employeeId": "EMP-001",
    "locationId": "LOC-NYC",
    "leaveType": "VACATION",
    "initialBalance": 10.0,
    "driftedBalance": 15.0,
    "driftAfterCallCount": 1
  }
}
```

Implementation:

```
async getBalance(employeeId, locationId, leaveType) {
  const key = `${employeeId}:${locationId}:${leaveType}`;
  const isDriftTarget = this.config.employeeId === employeeId &&
                        this.config.locationId === locationId &&
                        this.config.leaveType === leaveType;

  if (isDriftTarget && this.callCount >= this.config.driftAfterCallCount) {
    return { ...balance, availableBalance: this.config.driftedBalance };
  }

  this.callCount++;
  return balance;
}
```

Test Use:

- Drift detection testing
- Balance increase scenario (work anniversary)
- Balance decrease scenario (corrective adjustment)

4.5 CIRCUIT_BREAKER Behavior

Description: First N calls fail with 500 error, activating circuit breaker

Configuration:

```
{
  "behavior": "CIRCUIT_BREAKER",
  "config": {
    "failCount": 5,
    "errorCode": 500
  }
}
```

Implementation:

```
async getBalance(employeeId, locationId, leaveType) {
  if (this.callCount < this.config.failCount) {
    this.callCount++;
    throw new InternalServerError("HCM temporarily unavailable");
  }

  // After N failures, start succeeding (half-open state)
  const balance = this.dataStore.getBalance(employeeId, locationId, leaveType);
  if (!balance) throw new NotFoundError();
  return balance;
}
```

Test Use:

- Circuit breaker state transitions
- Circuit breaker recovery mechanism
- Fallback to cached data while circuit open

4.6 INVALID_DIMENSION Behavior

Description: Return 404 for specific employee-location-leaveType combo

Configuration:

```
{
  "behavior": "INVALID_DIMENSION",
  "config": {
    "invalidCombos": [
      { "employeeId": "EMP-001", "locationId": "LOC-INVALID", "leaveType":
"VACATION" },
      { "employeeId": "EMP-999", "locationId": "LOC-NYC", "leaveType": "SICK" }
    ]
  }
}
```

Implementation:

```
async getBalance(employeeId, locationId, leaveType) {
  const isInvalid = this.config.invalidCombos.some(combo =>
    combo.employeeId === employeeId &&
    combo.locationId === locationId &&
    combo.leaveType === leaveType
  );

  if (isInvalid) {
    throw new NotFoundError("Invalid employee-location combination");
  }

  const balance = this.dataStore.getBalance(employeeId, locationId, leaveType);
  if (!balance) throw new NotFoundError();
  return balance;
}
```

Test Use:

- Dimension validation error handling
- Invalid combo rejection

4.7 INSUFFICIENT_BALANCE Behavior

Description: Return 403 for requests exceeding balance

Configuration:

```
{
  "behavior": "INSUFFICIENT_BALANCE",
  "config": {
    "enforceStrictBalance": true
  }
}
```

Implementation (in submitTimeOffRequest):

```
async submitTimeOffRequest(request) {
  const balance = this.dataStore.getBalance(request.employeeId,
request.locationId, request.leaveType);

  if (balance.availableBalance < request.days) {
    throw new ForbiddenError("Insufficient balance: requested " + request.days +
                            ", available " + balance.availableBalance);
  }

  return {
    hcmRequestId: generateId(),
    status: "APPROVED",
    approvedAt: new Date().toISOString()
  };
}
```

Test Use:

- Insufficient balance error handling
- Edge cases (exact balance, > balance)

---

5. Data Store

5.1 Initial Data

```
{
  "employees": [
    {
      "employeeId": "EMP-001",
      "name": "Alice Johnson",
      "balances": [
        {
          "locationId": "LOC-NYC",
          "leaveType": "VACATION",
          "availableBalance": 10.0,
          "usedBalance": 5.0,
          "carryoverBalance": 2.0,
          "balanceYear": 2026,
          "lastUpdatedAt": "2026-04-24T10:00:00Z"
        },
        {
          "locationId": "LOC-NYC",
          "leaveType": "SICK",
          "availableBalance": 5.0,
          "usedBalance": 1.0,
          "carryoverBalance": 0.0,
          "balanceYear": 2026,
          "lastUpdatedAt": "2026-04-24T10:00:00Z"
        }
      ]
    },
    {
      "employeeId": "EMP-002",
      "name": "Bob Smith",
      "balances": [...]
    }
  ],
  "submittedRequests": [
    {
      "hcmRequestId": "HCM-001",
      "employeeId": "EMP-001",
      "locationId": "LOC-NYC",
      "leaveType": "VACATION",
      "days": 3.0,
      "startDate": "2026-05-01",
      "endDate": "2026-05-03",
      "status": "APPROVED",
```

```
      "submittedAt": "2026-04-24T10:00:00Z",
      "approvedAt": "2026-04-24T10:01:00Z"
    }
  ]
}
```

5.2 Seed Data Functions

```
// Seed 10 employees with various balance scenarios
seedTestData(): void {
  this.employees = [
    // Sufficient balance
    { employeeId: "EMP-001", locationId: "LOC-NYC", leaveType: "VACATION",
balance: 10 },
    { employeeId: "EMP-002", locationId: "LOC-NYC", leaveType: "VACATION",
balance: 15 },

    // Low balance
    { employeeId: "EMP-003", locationId: "LOC-NYC", leaveType: "VACATION",
balance: 2 },
    { employeeId: "EMP-004", locationId: "LOC-NYC", leaveType: "VACATION",
balance: 3 },

    // Zero balance
    { employeeId: "EMP-005", locationId: "LOC-NYC", leaveType: "VACATION",
balance: 0 },

    // Multiple leave types
    { employeeId: "EMP-006", locationId: "LOC-NYC", leaveType: "VACATION",
balance: 8 },
    { employeeId: "EMP-006", locationId: "LOC-NYC", leaveType: "SICK", balance: 5
},

    // Different locations
    { employeeId: "EMP-007", locationId: "LOC-NYC", leaveType: "VACATION",
balance: 10 },
    { employeeId: "EMP-007", locationId: "LOC-BOSTON", leaveType: "VACATION",
balance: 7 },
  ];
}
```

6. Testing with Mock HCM

6.1 Test Setup Example

```
describe('Time-Off Request Approval', () => {
  let mockHCMServer: MockHCMServer;
  let readyOnService: TimeOffRequestService;

  beforeAll(async () => {
    // Start mock HCM server
    mockHCMServer = new MockHCMServer();
    await mockHCMServer.start(3001);

    // Initialize ReadyOn service pointing to mock
    readyOnService = new TimeOffRequestService({
      hcmBaseUrl: 'http://localhost:3001/hcm'
    });
  });

  afterAll(async () => {
    await mockHCMServer.stop();
  });

  it('should handle timeout and retry', async () => {
    // Set mock to timeout behavior
    await mockHCMServer.setBehavior('TIMEOUT', { delayMs: 6000 });

    // ReadyOn should retry and eventually fail
    const request = await readyOnService.createRequest({
      employeeId: 'EMP-001',
      locationId: 'LOC-NYC',
      leaveType: 'VACATION',
      startDate: '2026-05-01',
      endDate: '2026-05-03',
      days: 3
    });

    expect(request.status).toBe('PENDING');

    // Attempt approval (should fail on HCM timeout)
    await expect(
      readyOnService.approveRequest(request.id)
    ).rejects.toThrow(HCMTimeoutException);
  });

  it('should detect drift', async () => {
```

```
    // Reset to normal
    await mockHCMServer.resetBehavior();

    // Create and approve request
    const request = await readyOnService.createRequest({ ... });

    // Simulate work anniversary: increase balance in HCM
    await mockHCMServer.updateBalance('EMP-001', 'LOC-NYC', 'VACATION', 15.0);

    // Batch sync should detect drift
    const driftResults = await readyOnService.detectDrift();
    expect(driftResults).toContainEqual({
      employeeId: 'EMP-001',
      locationId: 'LOC-NYC',
      leaveType: 'VACATION',
      previousBalance: 10,
      newBalance: 15,
      changeDetectedAt: expect.any(Date)
    });
  });
});
```

6.2 Behavior Transition Testing

```
it('should recover from circuit breaker', async () => {
  // Activate circuit breaker behavior
  await mockHCMServer.setBehavior('CIRCUIT_BREAKER', { failCount: 5 });

  // Make 5 calls, all should fail
  for (let i = 0; i < 5; i++) {
    await expect(
      readyOnService.getBalance('EMP-001', 'LOC-NYC', 'VACATION')
    ).rejects.toThrow();
  }

  // Circuit breaker should now be open
  // 6th call should also fail (circuit breaker blocks)
  await expect(
    readyOnService.getBalance('EMP-001', 'LOC-NYC', 'VACATION')
  ).rejects.toThrow(CircuitBreakerOpenException);

  // Reset behavior
  await mockHCMServer.resetBehavior();

  // Circuit breaker should now allow calls
  const balance = await readyOnService.getBalance('EMP-001', 'LOC-NYC',
'VACATION');
  expect(balance).toBeDefined();
});
```

---

7. Deployment

7.1 Docker Compose

```yaml
version: '3.8'

services:
  mock-hcm:
    build: ./mock-hcm
    ports:
      - "3001:3001"
    environment:
      NODE_ENV: test
      LOG_LEVEL: info
    volumes:
      - ./mock-hcm/data:/app/data
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3001/health"]
      interval: 10s
      timeout: 5s
      retries: 3

  time-off-service:
    build: .
    ports:
      - "3000:3000"
    environment:
      HCM_BASE_URL: http://mock-hcm:3001/hcm
      DATABASE_URL: sqlite:./data/timeoff.db
    depends_on:
      mock-hcm:
        condition: service_healthy
    volumes:
      - ./data:/app/data
```

7.2 Running Together

```bash
# Start both services
docker-compose up -d

# Check logs
docker-compose logs -f

# Stop services
docker-compose down
```

8. Admin Commands for Testing

```
# Reset to normal behavior
curl -X POST http://localhost:3001/hcm/admin/reset-behavior

# Activate timeout behavior
curl -X POST http://localhost:3001/hcm/admin/set-behavior \
  -H "Content-Type: application/json" \
  -d '{
    "behavior": "TIMEOUT",
    "config": { "delayMs": 6000 }
  }'

# Activate drift behavior
curl -X POST http://localhost:3001/hcm/admin/set-behavior \
  -H "Content-Type: application/json" \
  -d '{
    "behavior": "DRIFT",
    "config": {
      "employeeId": "EMP-001",
      "locationId": "LOC-NYC",
      "leaveType": "VACATION",
      "initialBalance": 10.0,
      "driftedBalance": 15.0,
      "driftAfterCallCount": 1
    }
  }'

# Manually update balance
curl -X POST http://localhost:3001/hcm/admin/update-balance \
  -H "Content-Type: application/json" \
  -d '{
    "employeeId": "EMP-001",
    "locationId": "LOC-NYC",
    "leaveType": "VACATION",
    "availableBalance": 20.0
  }'

# View current state
curl http://localhost:3001/hcm/admin/state | jq
```

9. Success Criteria

1. ✅ All 7 behaviors implemented and testable
2. ✅ Seed data includes 10 employees with various scenarios
3. ✅ Admin endpoints allow runtime behavior changes
4. ✅ All HCM endpoints return valid responses
5. ✅ Error responses match ReadyOn error handling expectations
6. ✅ Can run alongside Time-Off Service in docker-compose
7. ✅ Health check endpoint available
8. ✅ All responses include proper timestamps for sync testing